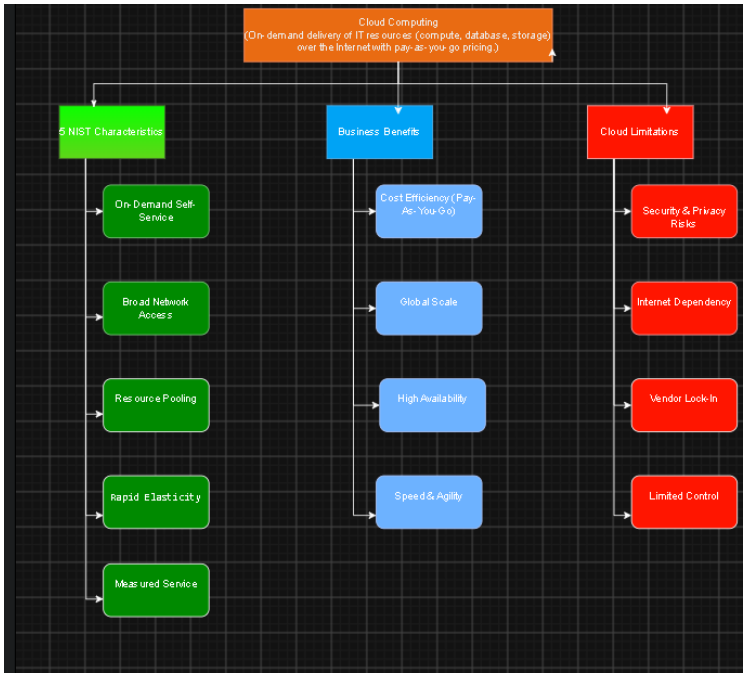
Course Name: Cloud Computing and Big Data Analytics

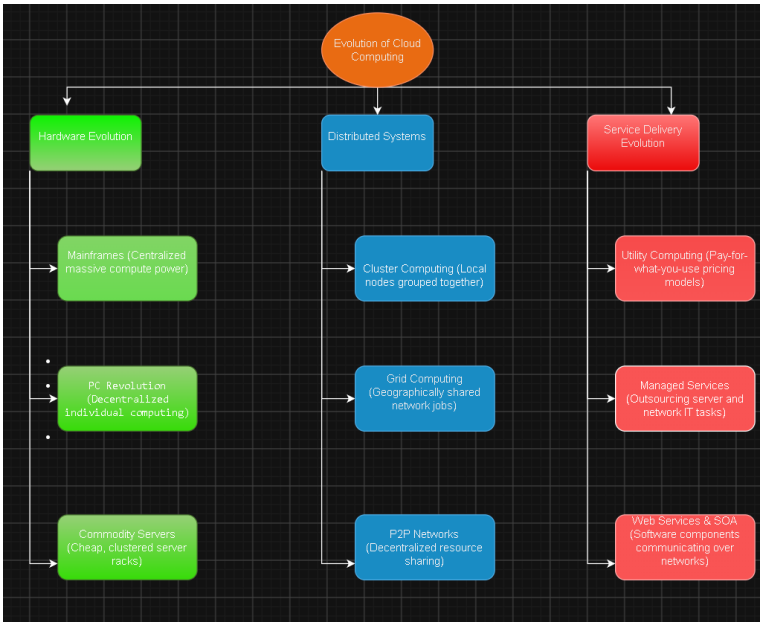
Course Code: CSA1504

Assessment: Assessment Tool 1: Concept Mapping

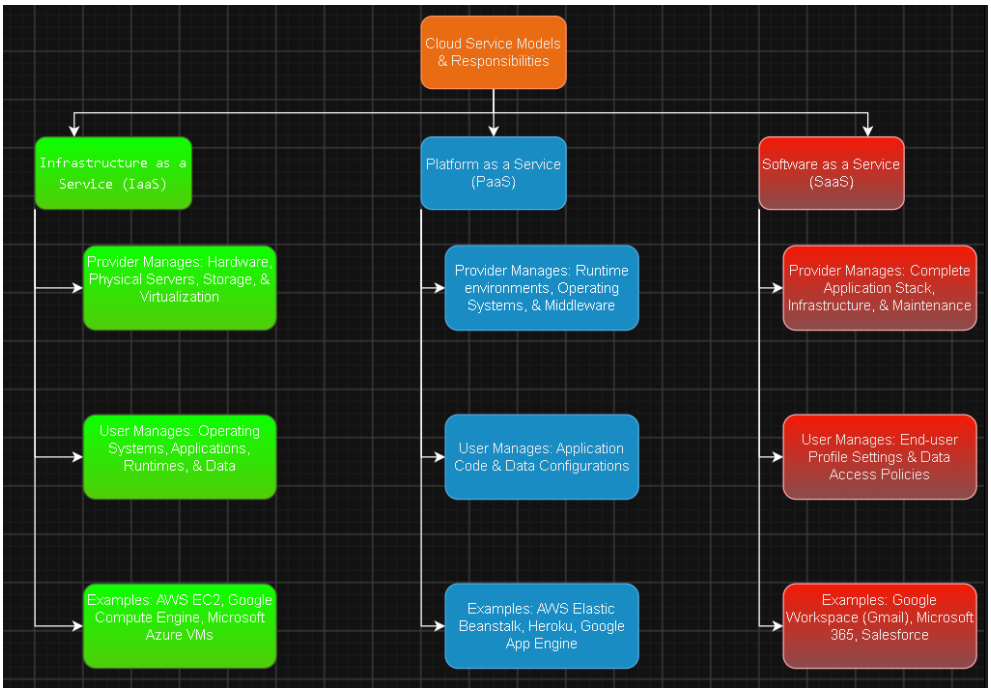
Name / Register Number: G. Ram Charan (192525043)



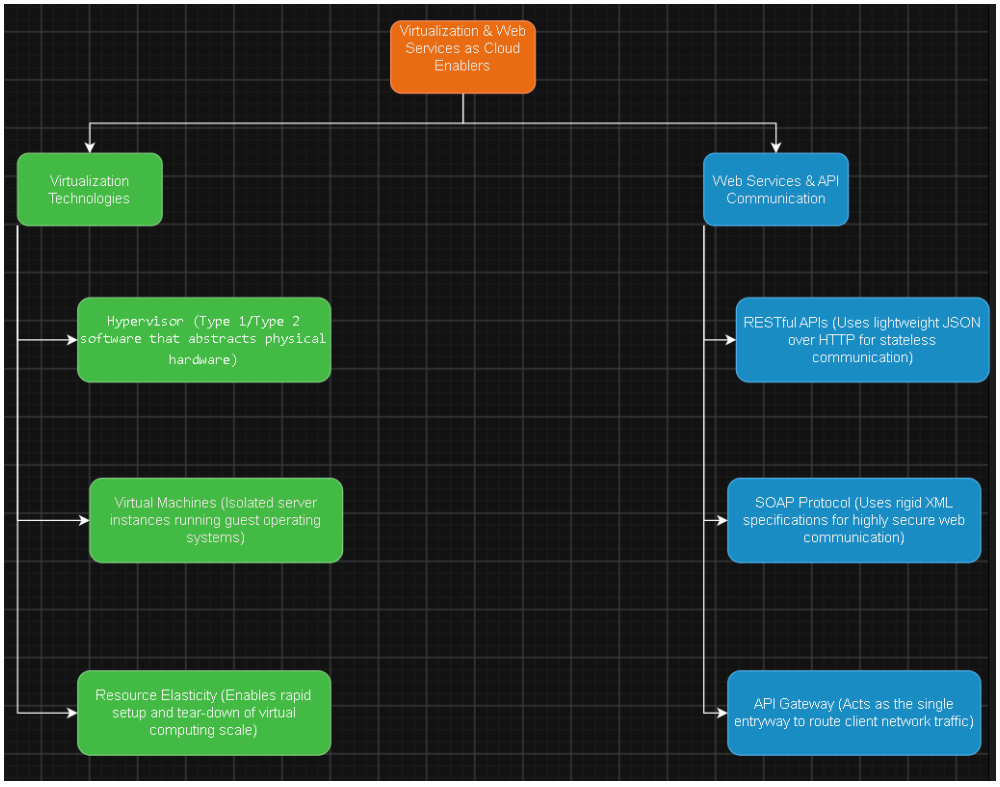
Concept Map 1: Cloud Computing Fundamentals



Concept Map 2: Evolution of Cloud Computing



Concept Map 3: Service Models



Concept Map 4: Virtualization and Web Services as Enablers

Service Model Responsibility Matrix

Service Category	Managed by Cloud Provider	Managed by Student Application Team	Why that Split Fits the Project
IaaS (Infrastructure as a Service)	Physical data center hosting, hardware provisioning, cooling, physical network infrastructure, and hypervisor hosts.	Operating system patching, custom network analysis environments (Python/Java runtimes), and application data packages.	Provides total control over physical computing configurations if long graph calculations require raw memory optimization tweaks.
PaaS (Platform as a Service)	Core runtime environments, automatic operating system patches, system middleware, and fundamental virtualization layers.	Python connectivity scripts, map location data configurations, and mathematical scaling adjustments.	**Highly recommended:** Frees the team from handling low-level operating system patches so they can focus on router placement algorithms.
SaaS (Software as a Service)	Entire application software stack, database engine management, user frontend updates, and computing layers.	High-level user profile access controls and graphical map dataset uploads.	Not suitable for deploying custom optimization algorithms directly, but ideal for external project monitoring dashboards.
FaaS / Serverless (Function as a Service)	Dynamic compute triggers, background runtime resource allocation, and instant capacity scaling engines.	Isolated analytical microservices (e.g., executing a single location's signal strength metrics).	Excellent for calculating localized network adjustments on demand whenever an engineer updates single node parameters.

Cloud Service Decision Matrix

System Module	Chosen Cloud Service	Technical Justification for the Project	Why that Split Fits the Project
Authentication Layer	**AWS Cognito / Firebase Authentication**	Provides secure user authentication and encryption so network engineers can log in safely without implementing custom cryptographic solutions.	Provides total control over physical computing configurations if long graph calculations require raw memory optimization tweaks.
Frontend Hosting	**Vercel / AWS Amplify**	Hosts the responsive network map interface globally using Content Delivery Networks (CDNs), ensuring fast loading times and high availability.	**Highly recommended:** Frees the team from handling low-level operating system patches so they can focus on router placement algorithms.
Backend / API Layer	**Node.js or Java on AWS Elastic Beanstalk (PaaS)**	Manages backend operations by receiving map boundary metrics and coordinating processing across analytics and optimization modules.	Not suitable for deploying custom optimization algorithms directly, but ideal for external project monitoring dashboards.
Database Management	**MongoDB Atlas / Amazon RDS**	Stores persistent spatial coordinates, building layouts, wireless channel information, and other application data required for analysis.	Excellent for calculating localized network adjustments on demand whenever an engineer updates single node parameters.

Technical Justification

CO1 Compliance Justification: Our architecture relies heavily on a Platform-as-a-Service (PaaS) framework to optimize operational management constraints. By leaning on managed services, the cloud provider handles virtualization abstractions, OS updates, and bare-metal server infrastructure, leaving our engineering team to safely manage structural source code, operational logic boundaries, and input data integrity.

Communication between modules is handled entirely by RESTful APIs, ensuring clean, decoupled data transmission from our frontend interactive dashboard to our backend calculation routines. System elasticity and scalability points are configured natively via auto-scaling rules tied directly to microservice instances. This design configuration allows computing components to provision more virtual resources during intense multi-node map calculations and scale down automatically when idle, fully realizing the efficiency and flexibility goals specified in the CO1 learning guidelines.